

Homework 0 – ECS 220, Winter 2026

Instructions

Collaboration. Unlike the other homeworks, this homework is *not collaborative*: each student must submit their own solution. High-level discussion is permitted as always, as described in the academic dishonesty policy in the course syllabus.

Purpose. This homework is intended to ensure that you have mastered (some of) the background knowledge for this course covered in the prerequisites ECS 120: undergraduate theory of computing and ECS 122a: undergraduate algorithms. If you did well in these two courses or their equivalents at another university, then these problems should be straightforward to solve. Conversely, if you are having difficulty with the problems, then you might consider waiting to take ECS 220 until you have gained proficiency in the subjects covered by those two courses.

If you have not taken these courses (for instance if you not a Computer Science major) but you fancy yourself a quick learner, I included links in each problem to relevant webpages teaching the background needed to solve them.

Submission. Your solutions to these problems are to be submitted on the Gradescope page for ECS 220 (see syllabus for link). See the syllabus for instructions on how to access the Gradescope page for this course. Each problem below appears as a separate “Assignment” in Gradescope, and they will each be graded individually on Gradescope. If you submit after the deadline above, Gradescope will automatically apply a late penalty as described in the syllabus.

The problems will be automatically graded. Furthermore, test cases are randomly generated each time you submit. It is unlikely for the same test case to be generated twice. Therefore, if your submission fails a test case, indicating a bug in your solution, for the sake of reproducing the bug, you should write down the test case so that, by doing “offline” testing on your own, you can figure out the bug before submitting again.

Gradescope will not automatically use your maximum score on any submission as your grade; instead it uses the most recent by default. To override this, on the bottom of the assignment page, click on *Submission History* to select which submission you want to “activate” (presumably you want this to be your highest score so far).

Randomized problems. The problems below marked **(randomized)** require interacting with Gradescope in a strange way. The idea is to have Gradescope randomly generate a problem, then you submit a solution, then Gradescope automatically grades it. Gradescope doesn’t support this workflow directly, so we will use the following hack to implement it:

For the randomized problems, you first *request* a problem by submitting as your first “solution” a file named `req` (no filename extension, so *don’t* name it `req.txt`). Gradescope will autograde

this as 0 points (don't take it personally), and then give text "feedback", which will contain the problem you are supposed to solve. On your *next* submission, you submit your solution to this problem, in a text file, which can have any name ending in the extension `.txt`. In particular make sure you do **not** include a file named `req` this time, or Gradescope will again give you 0 points and generate a new problem. Whatever score you get will replace the 0 that you got when submitting the `req` file.

Note that this "problem requesting" hack only applies to the **(randomized)** questions; for the others, simply submit your solution.

Problems

NFA 0's near end or 1 at end: Write a nondeterministic finite automaton with at most 10 states that decides whether its input is contained in the following set:

$$\{x \in \{0,1\}^* \mid x[|x| - 5] = x[|x| - 2] = 0 \text{ or } x[|x|] = 1\}.$$

We index strings starting at 1, so $x[1]$ is the first symbol, and $x[|x|]$ is the last symbol.

Format the automaton to be used with the simulator here: <http://web.cs.ucdavis.edu/~doty/automata/>. Your submission filename must end with the extension `.nfa`.

For help, see https://en.wikipedia.org/wiki/Nondeterministic_finite_automaton. I advise using the standard construction showing languages recognized by NFAs are closed under union; see Theorem 5.3.4 here: <https://canvas.ucdavis.edu/courses/192458/files/3918511/download>

TM binary to unary conversion: Write a Turing machine that converts its input from binary to unary. In other words, if the input is the string $x \in \{\$,0,1\}^*$, a $\$$ followed by the binary expansion of the nonnegative integer n (for example, $\$101$ to represent $n = 5$), then the machine outputs the string 1^n . You may assume the input string will be in this format, i.e., a $\$$ followed by either just a 0, or followed by a binary string beginning with 1. Format the Turing machine to be used with the simulator here: <http://web.cs.ucdavis.edu/~doty/automata/>. Click the Help link at the top of the page for instructions explaining how Turing machines process input and output in the simulator, and how to specify multi-tape Turing machines. Either a single-tape or a multiple-tape Turing machine may be used. Your submission filename must end with the extension `.tm`.

For help, see https://en.wikipedia.org/wiki/Turing_machine.

graph search: **(randomized)** Indicate the order in which nodes are visited in breadth-first and depth-first searches of undirected graphs. Graphs will be given in text format as an adjacency list, with nodes represented by letters. For example,

```
a: b,c,d
b: a,d
c: a
d: a,b
e: f
f: e
```

To break ties between neighbors of a single node, assume the first visited is in alphabetical order. If the graph is not connected, then give the traversal order of the connected component containing the node **a**. In this example, a BFS starting at node **a** would visit the nodes in order **a, b, c, d**, (leaving **e** and **f** unvisited, since they are in a different connected component than **a**) and a DFS would visit them in order **a, b, d, c**. To indicate this, upload a text file that has these orders on two lines (BFS order on line 1 and DFS order on line 2), with nodes separated by spaces on a line. For example:

```
a b c d
a b d c
```

For help, see https://en.wikipedia.org/wiki/Breadth-first_search and https://en.wikipedia.org/wiki/Depth-first_search.

rank time bounds: (randomized) Rank various time bounds (functions of input size n , e.g., n^2 , 2^n) in order of their asymptotic growth rates. The functions will be given as L^AT_EX-like code, e.g.,

```
(0) log n
(1) 2 log n
(2) 1
(3) 2^n
```

You will rank them by uploading a text file with the proper nondecreasing order of asymptotic growth rates. Assume all logarithms are base 2. Some time bounds may have the same asymptotic growth rate, in which case listing them in either relative order would be considered correct. More precisely, a correct ordering $f_1, f_2, \dots$ satisfies $f_i = O(f_{i+1})$ for all i . In the above example, since $1 = o(\log n)$, $\log n = \Theta(2 \log n)$, and $\log n = o(2^n)$, there are two correct solutions:

```
2 0 1 3
```

and

```
2 1 0 3
```

For help, see https://en.wikipedia.org/wiki/Big_O_notation.